Software Requirements Specification

For

Rack-Level Micro-Climate and Growth Optimizer for Oyster Mushroom Farms

Project ID: R26-ISE-002

B.Sc. (Hons) in Information Technology Specializing in Information Systems Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

Table of Contents

Table of Contents **2**
Table of Figures **3**
Table of Tables **3**
Revision History **4**
1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 2
1.4 Product Scope 2
1.5 References 3
2. Overall Description **3**
2.1 Product Perspective 3
2.2 Product Functions 4
2.3 User Classes and Characteristics 7
2.4 Operating Environment 8
2.5 Design and Implementation Constraints 9
2.6 User Documentation 10
2.7 Assumptions and Dependencies 11
3. External Interface Requirements **12**
3.1 User Interfaces 12
3.2 Hardware Interfaces 27
3.3 Software Interfaces 28
3.4 Communications Interfaces 29
4. System Features **31**
4.1. Moisture Stress Regulation Using Temperature-Humidity 31
4.2 Disease Detection & Rack-Level Tracking 32
4.3 Biological Stage Detection & Harvest Time Calculation 35
5. Other Non-functional Requirements **36**
5.1 Performance Requirements 36
5.2 Safety Requirements 36
5.3 Security Requirements 37
5.4 Software Quality Attributes 37
5.5 Business Rules 38
6. Other Requirements **39**
Appendix A: Glossary **39**
Appendix B: Analysis Models **40**
State –Transition Diagram 40
Data Flow Diagram 41
Appendix C: To Be Determined List **41**

Table of Figures

Figure 1 Sequence Diagram of Disease Detection Component 6
Figure 2 Main Dashboard Page 13
Figure 3 Create Account Page 14
Figure 4 Log in Page 15
Figure 5 Settings Page 16
Figure 6 Forgot Password Page 17
Figure 7 Reset Password Page 18
Figure 8 Disease Detection Main Page 19
Figure 9 Disease Detection History Page 20
Figure 10 Disease Detection Tracking Page 21
Figure 11 Report Export Successful Page 22
Figure 12 Unsupported File Format Error Page 23
Figure 13 Image too Large Error Page 24
Figure 14 Rack ID Required Validation Page 25
Figure 15 Low Image Quality Error Page 26
Figure 16 Detection Failed Error Page 27
Figure 17 State Transition Diagram of the System 40
Figure 18 Data Flow Diagram of the System 41

Table of Tables

Table 1 Communication Interfaces 30
Table 2 Use Case of Moisture Stress Regulation Component 32
Table 3 Stimulus/Response Sequences of Disease Detection Component 33
Table 4 Use Case 1 of Disease Detection Component 34
Table 5 Use Case 2 of Disease Detection Component 35
Table 6 Use Case 3 of Disease Detection Component 35
Table 7 Stimulus/Response Sequences of Biological Stage Detection Component 36
Table 8 Other Requirements 39
Table 9 Glossary 40

Revision History

Name	Date	Reason For Changes	Version

Project Name	Oyster Mushroom Suraki
Document Prepared By	M. I. Amana (IT22156624), H. P. D. M Chandrasiri (IT22180148), R.T. S. Dissanayake (IT22062406)
Document Reviewed and Updated By	M. I. Amana (IT22156624)
Document Version	1.0
Date	13th May 2026
Nature	External - Confidential

1. Introduction

1.1 Purpose

This SRS document specifies the software requirements for the first version (Version 1.0) of the system, focusing on the mobile application, backend services, AI-based image processing modules, and IoT data integration mechanisms. The purpose of this Software Requirements Specification (SRS) document is to define the complete functional and non-functional requirements for the proposed Rack-Level Micro-Climate & Growth Optimizer for Oyster Mushroom Cultivation. This document serves as the primary guideline for the design, development, implementation, testing, and evaluation of the system while establishing a common understanding among the research team, academic supervisors, and relevant stakeholders. Currently, oyster mushroom farmers in Sri Lanka face significant challenges in maintaining optimal environmental conditions throughout different biological growth stages due to reliance on manual monitoring and trial-and-error methods. Variations in humidity, temperature, and CO_2 concentration across cultivation racks create unstable micro-climates that directly impact mushroom growth, yield quality, and disease occurrence. To address these challenges, the proposed system introduces an intelligent smart farming solution integrating IoT technologies, computer vision, and mobile computing. The system is built around four main components:

- Disease Detection and Rack-Level Tracking
- Growth Stage Detection and Harvest Time Calculation
- Moisture Stress Monitoring using Temperature-Humidity
- Physiological CO_2 Stress Prediction

1.2 Document Conventions

This SRS document follows standardized conventions to ensure clarity, consistency, and ease of understanding for all stakeholders involved in the development of the proposed Rack-Level Micro-Climate & Growth Optimizer for oyster mushroom cultivation. The structure and formatting of the document are aligned with IEEE 830 / IEEE 29148 Software Requirements Specification standards, which are widely accepted in academic and professional software engineering practices. The document is organized using numbered sections and subsections, where major sections are represented using bold numbered headings (e.g., 1. Introduction, 2. Overall Description), while sub-sections follow a hierarchical numbering structure (e.g., 1.1 Purpose, 1.2 Document Conventions) to support clear referencing and traceability. Requirement statements use standardized keywords such as “shall” to indicate mandatory system requirements, “should” for recommended functionalities, and “may” for optional features. Use cases are documented using a standardized format including actors, preconditions, main success scenarios, alternative flows, and postconditions. UML models and diagrams follow standard UML notation to represent workflows and interactions within the system. All requirements specified in this document are considered high priority unless otherwise stated, and any future modifications must follow a formal change management process to maintain system integrity and consistency.

1.3 Intended Audience and Reading Suggestions

This SRS document is intended for a diverse group of stakeholders involved in the design, development, implementation, deployment, and academic evaluation of the proposed Rack-Level Micro-Climate & Growth Optimizer for oyster mushroom cultivation. Each stakeholder group will use the document for different purposes; therefore, specific sections are designed to address their respective responsibilities and interests. The primary intended audiences include:

- Software Developers & AI Engineers: Who will use the document to understand system functionalities, IoT device integration, backend API communication, database operations, and implementation details related to machine learning, computer vision, and environmental monitoring modules.
- Academic Supervisors & Examination Panel: Who will assess the technical depth, research contribution, architectural design, and alignment of the project with the academic requirements of the BSc. in Information Systems Engineering degree program at SLIIT.
- Agricultural Industry Stakeholders & Oyster Mushroom Farmers: Who will evaluate the practicality, reliability, and effectiveness of the system in improving oyster mushroom cultivation.
- Testers & Quality Assurance (QA) Engineers: Who will derive system test cases and validation scenarios from the documented requirements, focusing on sensor accuracy, backend API reliability, machine learning prediction accuracy, and real-time environmental control performance.
- Documentation Writers & Researchers: Who will utilize the SRS document to prepare user manuals, technical reports, deployment guides, academic publications, and future research documentation related to smart agriculture and IoT-based farming systems.

1.4 Product Scope

The scope of the proposed Rack-Level Micro-Climate & Growth Optimizer is to design and develop an intelligent smart farming system that enhances oyster mushroom cultivation in Sri Lanka by enabling real-time environmental monitoring, automated control, and AI-based decision support. The system focuses on improving productivity, reducing crop loss, and minimizing manual intervention through the integration of IoT, mobile computing, and machine learning technologies.

The project includes the development of an IoT-based sensing system using ESP32 and DHT11 to continuously monitor key environmental parameters such as temperature and humidity at rack level. It also includes a backend system developed using FastAPI to process sensor data, manage API communication, and coordinate system operations.

A mobile application developed using React Native is included to provide farmers with real-time dashboards, alerts, and control capabilities for managing mushroom farm conditions. The system also incorporates AI-based modules for mushroom growth stage detection and disease identification using computer vision techniques.

The scope further covers automated control of environmental devices such as mist sprayers and exhaust fans based on sensor readings and predefined thresholds, ensuring optimal growing conditions for oyster mushrooms.

However, the system does not include large-scale commercial farm automation infrastructure or fully autonomous harvesting mechanisms. It is primarily designed for small and medium-scale mushroom farmers and focuses on rack-level environmental optimization rather than full agricultural supply chain management.

1.5 References

The following documents and resources were referred to during the preparation of this Software Requirements Specification (SRS) and are relevant for understanding the system design, implementation, and standards:

- "HORDI Crop – Mushroom – Department of Agriculture Sri lanka." https://doa.gov.lk/hordi-crop-mushroom/
- "29148-2018 - ISO/IEC/IEEE International Standard - Systems and software engineering -- Life cycle processes -- Requirements engineering," IEEE Standard | IEEE Xplore, Nov. 30, 2018. https://ieeexplore.ieee.org/document/8559686

2. Overall Description

2.1 Product Perspective

The Oyster Mushroom Disease Detection and Growth Optimization System is a new, self-contained software product developed to support oyster mushroom farmers in monitoring, analyzing, and optimizing farm conditions.

This system is not a continuation of an existing product nor a direct replacement for a specific legacy system. Instead, it is designed as an integrated platform that combines multiple functionalities into a single application. The system consists of four primary components, Disease Detection and Rack-Level Tracking, Growth Stage Detection, Moisture Stress Monitoring, and CO_2 Stress Monitoring.

The Disease Detection component uses image-based analysis to identify infections such as green mold and black mold, while also enabling rack-level tracking of contamination. The Growth Stage Detection component analyzes oyster mushroom development stages to assist in determining optimal harvest times. The Moisture Stress Monitoring component uses temperature and humidity data from IoT sensors to detect unfavorable environmental conditions, and the CO_2 Stress Monitoring component evaluates gas concentration levels to identify stress affecting mushroom growth.

The system operates primarily as a mobile-based application used by farmers, supported by backend services that handle data processing, storage, and communication. The backend system integrates machine learning models for image analysis and processes sensor data received from IoT devices. External hardware components, such as temperature, humidity, and CO_2 sensors connected via microcontrollers (e.g., ESP32), provide real-time environmental data to the system.

In addition to internal components, the system interacts with external services such as cloud storage platforms and databases for data persistence. Communication between components is facilitated through standard web protocols (HTTP/HTTPS), ensuring smooth data exchange between the mobile application, backend services, and IoT devices.

2.2 Product Functions

2.2.1 Moisture Stress Regulation Using Temperature-Humidity

The rack-level moisture stress regulation component helps to keep moisture levels at a desired level in the mushroom growing racks by monitoring moisture levels inside the racks based on temperature and humidity readings acquired by sensors. This component is not just based on fixed humidity but considers the relationship between the temperature and the relative humidity to determine effective moisture stress. This is important because the same moisture content may have different effects on the growth of mushrooms at various temperatures. The component contributes to the alleviation of moisture related problems in cultivation like cap drying, pin abortion, fruit body development irregularity, and yield irregularity.

The main functions of Moisture Stress regulation component include:

- Gather the temperature and relative humidity information at the rack level from sensors.
- Keep environmental readings with respective rack and cultivation batch.
- Discuss the interaction between temperature and humidity and recognize moisture stress.
- Compute a temperature-humidity based Transpiration Stress Index.
- Determine if the rack is at a correct moisture level, low moisture stress, or high moisture stress.
- Create moisture alerts in a rack-wise manner if stress values are beyond the acceptable limits.
- Provide corrective measures like using a mist sprayer, exhaust fan activation, etc.

2.2.2 Disease Detection and Rack-Level Tracking

The Disease Detection component of the system provides functionality to identify and manage infections in oyster mushroom farms using image-based analysis. This component enables users to capture or upload images of oyster mushroom bags, analyze them using an AI-based detection model, and receive actionable insights for disease management.

The major functions of the Disease Detection component include:

- Capturing images of oyster mushroom bags using a mobile device camera
- Uploading images from the device gallery for analysis
- Validating image inputs for format, size, and quality before processing
- Processing images using a trained YOLOv8 model for disease detection
- Identifying and classifying diseases such as green mold (Trichoderma) and black mold (Aspergillus Niger)
- Displaying detection results
- Allowing users to input and associate Rack ID and Bag ID with detection results
- Storing detection records in the system database with timestamps
- Displaying historical detection records with filtering and search capabilities
- Supporting rack-level tracking of infected and healthy oyster mushroom bags
- Allowing users to export detection reports
- Handling errors and invalid inputs during image upload and processing

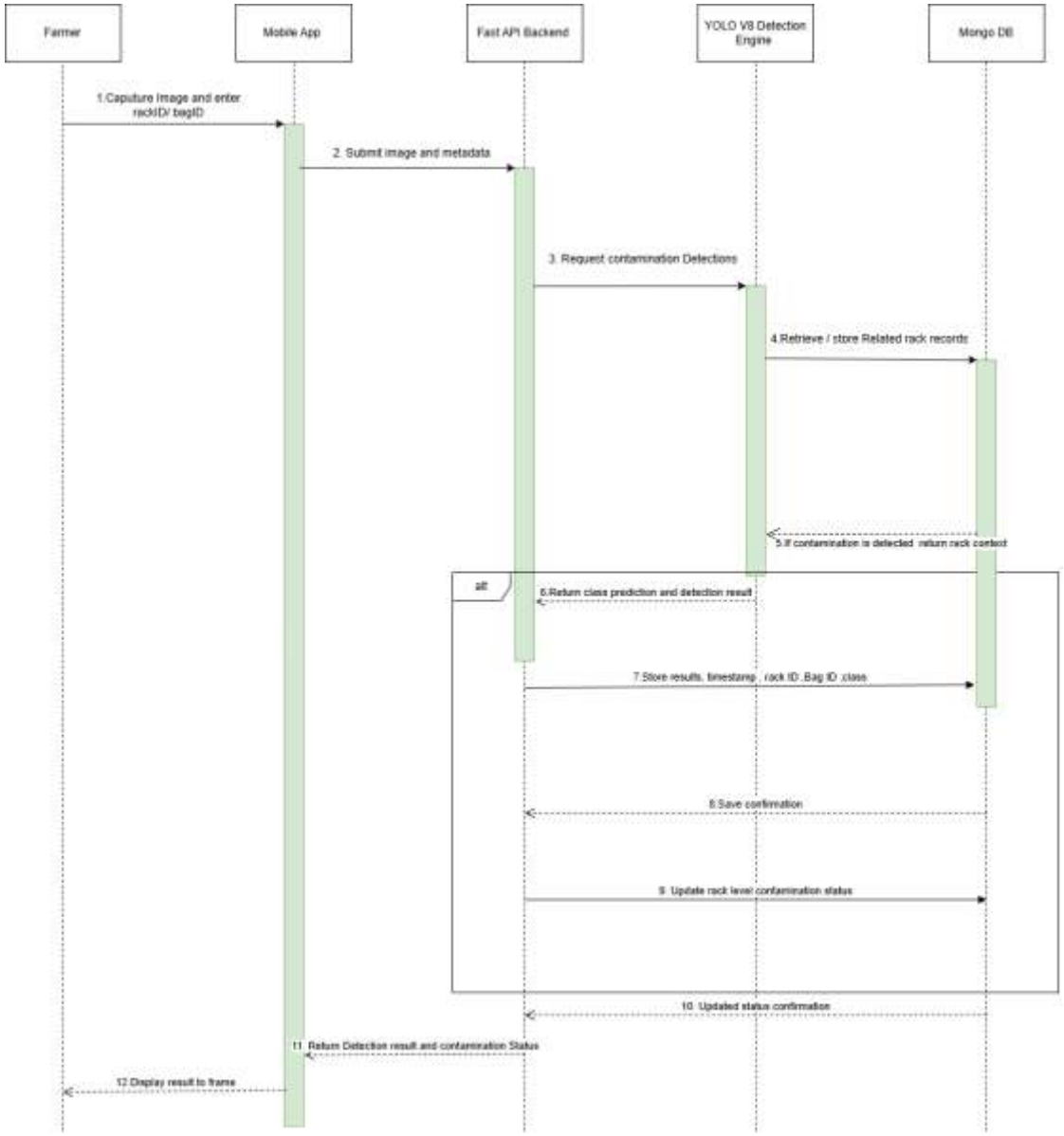


Figure 1 Sequence Diagram of Disease Detection Component

2.2.3 Biological Stage Detection and Harvest Prediction

The Biological Stage Detection component of the system provides functionality to identify the current growth stage of oyster mushrooms and estimate the remaining time until harvest using image-based analysis and machine learning techniques. This component enables farmers to capture or upload mushroom images, analyze them using an AI-based stage detection model, and receive clear visual information about mushroom development and harvest readiness.

The major functions of the Biological Stage Detection component include:

- Capturing images of oyster mushroom fruiting bodies using a camera in the portable device.
- Validating image inputs for format, resolution, and image quality before processing.
- Processing images using a trained YOLOv8-based machine learning model for biological stage detection.
- Identifying and classifying oyster mushroom growth stages such as early growth, pinning, mature, and ready-to-harvest stages.
- Estimating the remaining number of days until harvesting based on detected biological features and growth patterns.
- Displaying stage detection and harvest estimation results through the mobile application dashboard.
- Storing biological stage records in the system database with timestamps for historical tracking.
- Handling invalid images, processing failures, and system errors during image upload and analysis operation.

2.3 User Classes and Characteristics

The Oyster Mushroom Disease Detection and Growth Optimization System is designed to support multiple user classes with varying levels of technical expertise, responsibilities, and interaction frequency. These user classes are differentiated based on their usage patterns, access levels, and the specific system components they interact with, including Disease Detection, Growth Stage Detection, Moisture Stress Monitoring, and CO_2 Stress Monitoring.

The primary user class is Farmers, who are the main end users of the system. Farmers are expected to use the system frequently for daily monitoring and decision-making activities. They interact with all major components of the system, including capturing and uploading oyster mushroom images for disease detection, monitoring environmental conditions, and tracking rack-level data. This user class may have limited technical expertise; therefore, the system is designed with a simple, intuitive, and user-friendly interface to ensure ease of use. Farmers are considered the most important user class, as the system is primarily developed to support their needs and improve their productivity and outcomes.

A secondary user class is Farm Supervisors or Farm Managers, who may oversee multiple racks or farming operations. These users are expected to have moderate technical knowledge and will primarily use the system for monitoring, analysis, and decision-making at a broader level. They may rely more heavily on features such as historical data analysis, report generation, and environmental monitoring (moisture and CO_2 stress). Supervisors may also validate detection results and ensure proper system usage across the farm.

Another user class includes System Administrators or Technical Users, who are responsible for system maintenance, configuration, and monitoring of system performance. These users typically have a higher level of technical expertise and may interact with backend components, manage user accounts, monitor system logs, and ensure the proper functioning of IoT devices and data flow. Their interaction with the system is less frequent compared to farmers but is critical for system reliability and maintenance.

Additionally, Researchers or Developers may use the system for experimental and analytical purposes. This user class may focus on evaluating model performance, improving detection accuracy, and analyzing collected data. They may interact with the system at a deeper technical level, particularly with the AI models and datasets, but are not considered primary operational users.

Each user class has different access levels and requirements. Farmers and supervisors require full access to functional features, while administrators require extended privileges for system management. Researchers and developers require access to analytical data and system outputs.

2.4 Operating Environment

The Oyster Mushroom Disease Detection and Growth Optimization System is designed to operate within a hybrid environment that combines mobile devices, cloud-based services, and IoT hardware components. The system is primarily intended for use by farmers through mobile applications, while backend processing and data storage are managed through server-side and cloud infrastructure.

The user interface of the system will operate on smartphones, supporting commonly used mobile operating system, Android. Application is expected to run on modern devices with standard hardware capabilities, including a functional camera for capturing oyster mushroom images and sufficient processing power to handle user interactions smoothly. The interface is optimized for mobile screen sizes to ensure usability and accessibility.

The backend system will operate on a server environment, which may be deployed on local servers or cloud platforms. The backend is expected to run on operating systems such as Windows Server and will support Python-based frameworks such as FastAPI for handling API requests, processing data, and managing communication between system components. The system relies on cloud-based or remote databases such as MongoDB for data storage and retrieval.

The Disease Detection and Growth Stage Detection components require an environment capable of supporting machine learning model execution. This may involve GPU-enabled environments during model training, while inference may be handled on standard server hardware or optimized for edge deployment where applicable.

The Moisture Stress Monitoring and CO_2 Stress Monitoring components depend on IoT hardware platforms, including sensors for temperature, humidity, and CO_2 levels, connected through microcontrollers such as ESP32 or similar devices. These devices operate within the farm environment and transmit data to the backend system via wireless communication protocols such as Wi-Fi.

The system requires a stable network connection to support communication between mobile devices, IoT components, and backend services. Data exchange is expected to occur over HTTP/HTTPS protocols, with JSON used as the primary data format. While limited functionality may be available offline, full system capabilities, including real-time monitoring and cloud-based processing, depend on internet connectivity. The software is expected to coexist with other standard applications on user devices without causing conflicts. It should operate efficiently alongside other mobile applications, ensure minimal resource consumption and avoiding interference with device performance.

2.5 Design and Implementation Constraints

The design and implementation of the Oyster Mushroom Disease Detection and Growth Optimization System are subject to several constraints that may limit the options available to developers and influence system architecture and performance. The system is constrained by the use of specific technologies and tools.

The Disease Detection and Growth Stage Detection components rely on object detection frameworks, which requires sufficient computational resources for model execution. Hardware limitations also impose constraints on the system. The IoT-based Moisture Stress Monitoring and CO_2 Stress Monitoring components depend on sensors for temperature, humidity, and CO_2 levels. These sensors must be compatible with microcontrollers such as ESP32 or similar devices. Limitations in sensor accuracy, calibration, and data transmission frequency may affect system performance and the precision of monitoring and predictions.

The system is designed to operate primarily on mobile devices, which introduces constraints related to device performance, screen size, and battery usage. Image processing tasks, especially those related to oyster mushroom disease detection and growth stage analysis, may require optimization to ensure acceptable response times on devices with limited processing power.

The system depends on internet connectivity for key operations such as image processing, data synchronization, and cloud storage. Therefore, network availability and bandwidth limitations may affect system responsiveness and functionality. Communication between system components is expected to follow standard protocols such as HTTP/HTTPS, with data exchanged in JSON format.

Security considerations also impose constraints on the system design. User data, including personal information and farm-related records, must be securely stored and transmitted. Authentication mechanisms must be implemented to restrict unauthorized access, and secure communication protocols must be used to protect data integrity and privacy.

Design conventions and usability requirements further influence the system. The user interface must follow a consistent, minimal, and nature-inspired design suitable for users with low technical expertise. This includes standardized layouts, color schemes, and interaction patterns across all components, including Disease Detection, Growth Stage Detection, Moisture Stress Monitoring, and CO_2 Stress Monitoring.

Additionally, the system may need to comply with general software development standards and best practices, including modular design, maintainability, and scalability. If future maintenance or enhancements are required, the system architecture must support easy updates and integration of additional features. Any changes in these constraints, such as limitations in hardware availability, changes in technology stacks, or variations in network conditions, may impact the design decisions and overall system performance.

2.6 User Documentation

The Oyster Mushroom Disease Detection and Growth Optimization System will be supported by a set of user documentation materials designed to assist users in understanding, operating, and effectively utilizing the system. These documents are intended for farmers and other end users with varying levels of technical knowledge, and therefore emphasize clarity, simplicity, and practical guidance.

- A User Manual will be provided as the primary documentation component. This manual will include step-by-step instructions on how to use the system, covering all major features such as Disease Detection, Growth Stage Detection, Moisture Stress Monitoring, and CO_2 Stress Monitoring. The manual will also include explanations of system outputs, such as color-coded indicators and alert messages.
- An Installation and Setup Guide will be included to assist users in setting up the system. This will cover application installation, account creation, and configuration of IoT components such as temperature, humidity, and CO_2 sensors. It will provide instructions on proper sensor placement and connectivity to ensure accurate data collection for the monitoring components.

- An On-screen Help and Guidance System will be integrated into the application to provide real-time assistance. This includes inline validation messages that guide users while performing actions such as entering Rack IDs, uploading images, etc. Error messages and feedback prompts are designed to be clear and actionable to support user decision-making.
- A Quick Start Guide will be provided as a simplified reference document for first-time users. This guide will highlight the basic steps required to begin using the system, including logging in, accessing the dashboard, and performing initial tasks.

The documentation will be delivered in digital format (PDF). The content will follow standard documentation practices, using simple language, structured sections, and visual aids such as screenshots where necessary. All user documentation components are designed to ensure that users can effectively interact with the system and fully utilize all four core components without requiring advanced technical knowledge.

2.7 Assumptions and Dependencies

The development and operation of the Oyster Mushroom Disease Detection and Growth Optimization System are based on several assumptions that may influence the system requirements. It is assumed that users will have access to a smartphone and internet, and that they possess basic digital literacy to navigate the application and interact with its features effectively.

- For the Disease Detection component, it is assumed that images captured or uploaded by users are of sufficient quality, including proper lighting, focus, and framing, to allow accurate detection by the YOLOv8 model. The system also assumes that the trained model is capable of reliably identifying common oyster mushroom diseases such as green mold and black mold under real-world farm conditions. The accuracy of this component is dependent on the quality and diversity of the training dataset.
- For the Growth Stage Detection component, it is assumed that the visual features of oyster mushrooms at different growth stages can be accurately identified through image processing techniques. It is also assumed that the dataset used for training includes sufficient variation across different stages, enabling the model to generalize effectively in real-world scenarios.
- For the Moisture Stress Monitoring component, it is assumed that environmental data such as temperature and humidity are accurately captured through IoT sensors. The system depends on correct sensor placement, calibration, and consistent data transmission. Any inaccuracies in sensor readings may affect stress analysis and recommendations.
- For the CO_2 Stress Monitoring component, it is assumed that CO_2 sensors provide reliable measurements of gas concentration within the growing environment. The system assumes that CO_2 levels have a measurable impact on oyster mushroom growth and that these values can be used to predict stress conditions effectively.

The system depends on a stable internet connection for key functionalities such as image processing, real-time monitoring, data synchronization, and cloud storage. While limited offline capabilities may be supported, full system functionality depends on network availability. The project also depends on external software components and technologies, including object detection frameworks to backend APIs (such as FastAPI), and database systems (such as MongoDB). The performance, compatibility, and availability of these third-party components are critical to system operation. Any deviation from these assumptions, including poor image quality, unreliable sensor data, insufficient training data, lack of internet connectivity, or failure of external components, may impact system performance and the validity of the requirements specified in this document.

3. External Interface Requirements

3.1 User Interfaces

The system provides a mobile-based graphical user interface (GUI) designed with a clean, minimal, and nature-inspired theme to ensure usability for farmers with varying levels of technical expertise. The interface uses a consistent soft green and beige color palette, rounded components, and intuitive navigation to enhance readability and ease of interaction. The application follows a component-based UI structure, where each major functionality (Disease Detection, Growth Stage Detection, Moisture Stress Monitoring, CO_2 Stress Monitoring) is accessible through a centralized dashboard.



Figure 2 Main Dashboard Page

- The main dashboard provides an overview of farm conditions and quick access to system components. It displays environmental data (temperature and humidity) and four primary modules, Disease Detection, Growth Stage Detection, Moisture Stress Monitoring and CO_2 Stress Monitoring.

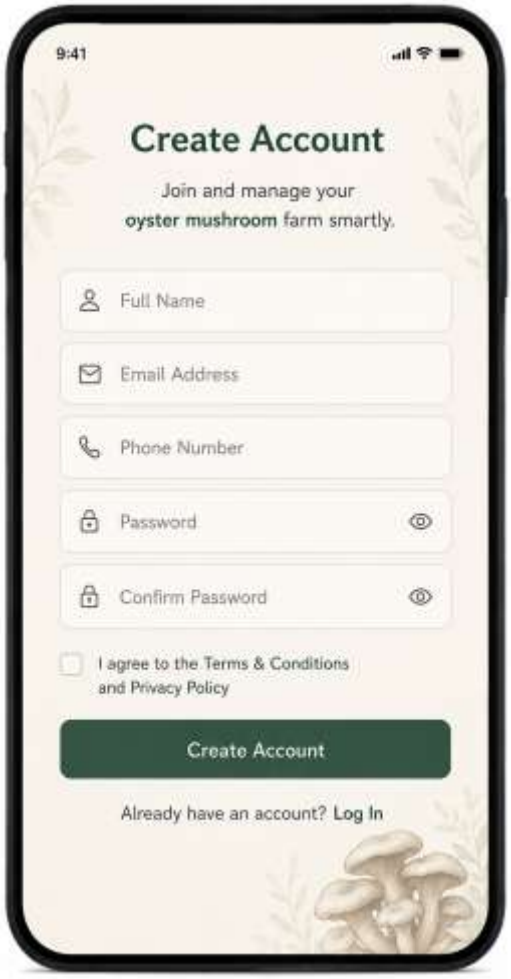


Figure 3 Create Account Page

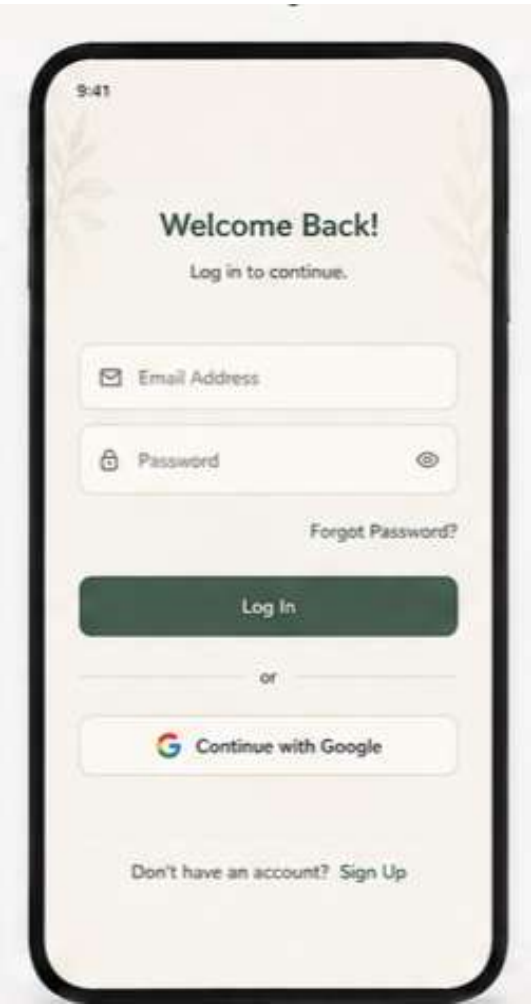


Figure 4 Log in Page

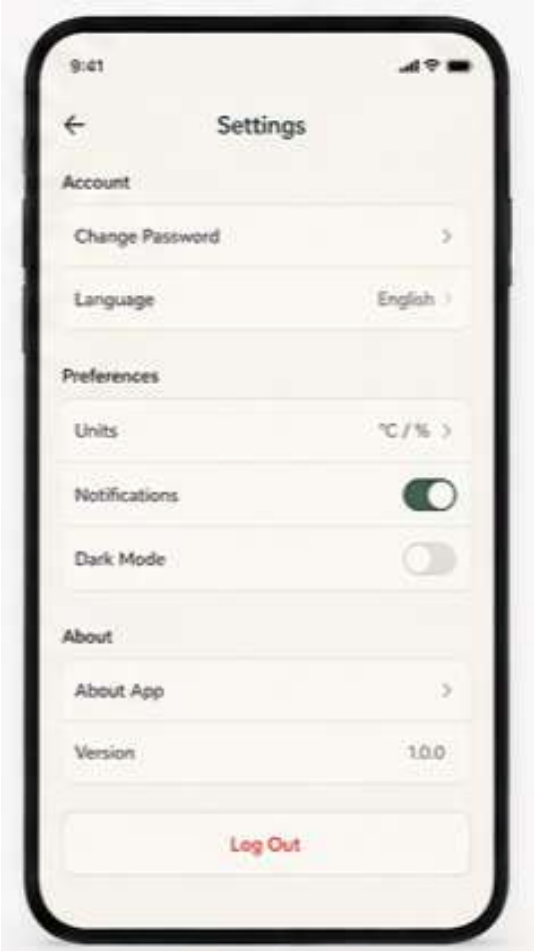


Figure 5 Settings Page

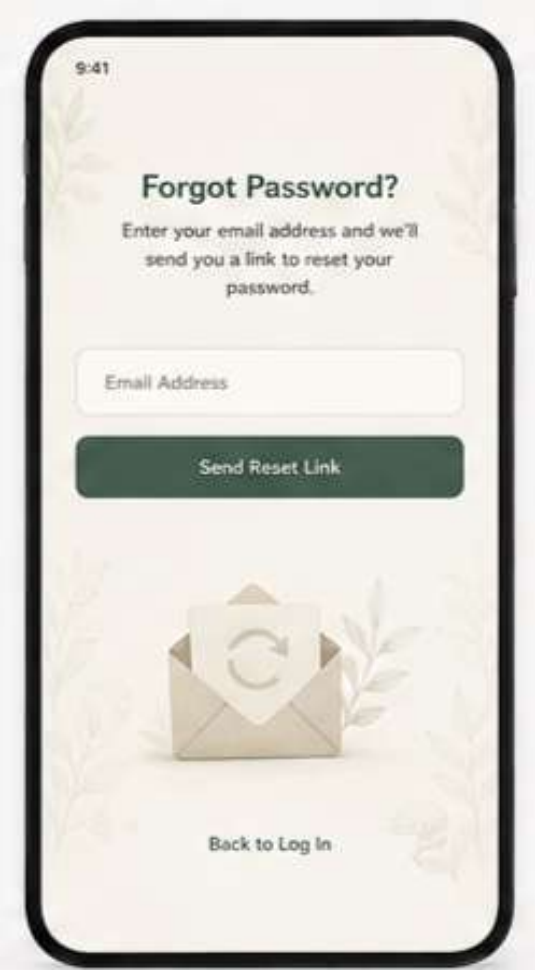


Figure 6 Forgot Password Page

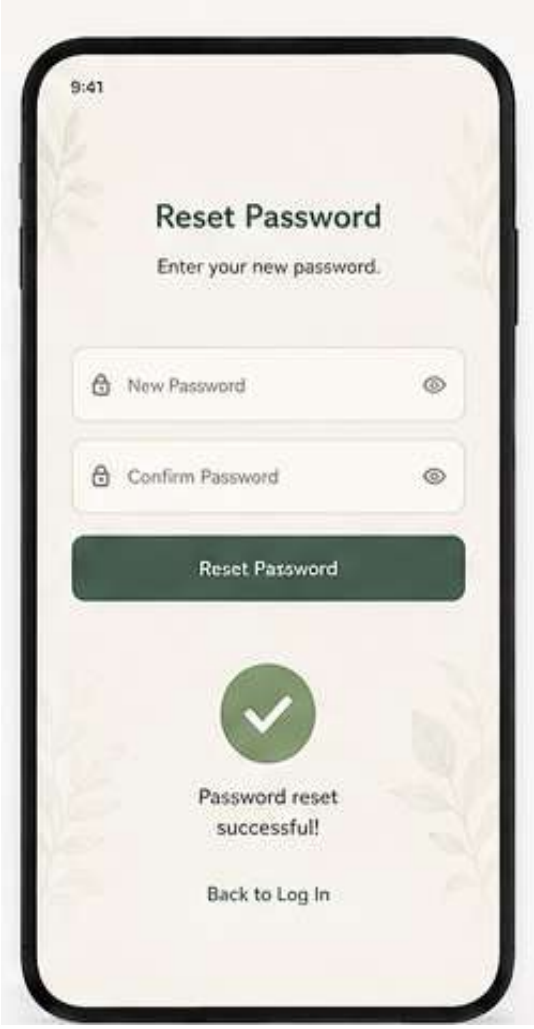


Figure 7 Reset Password Page

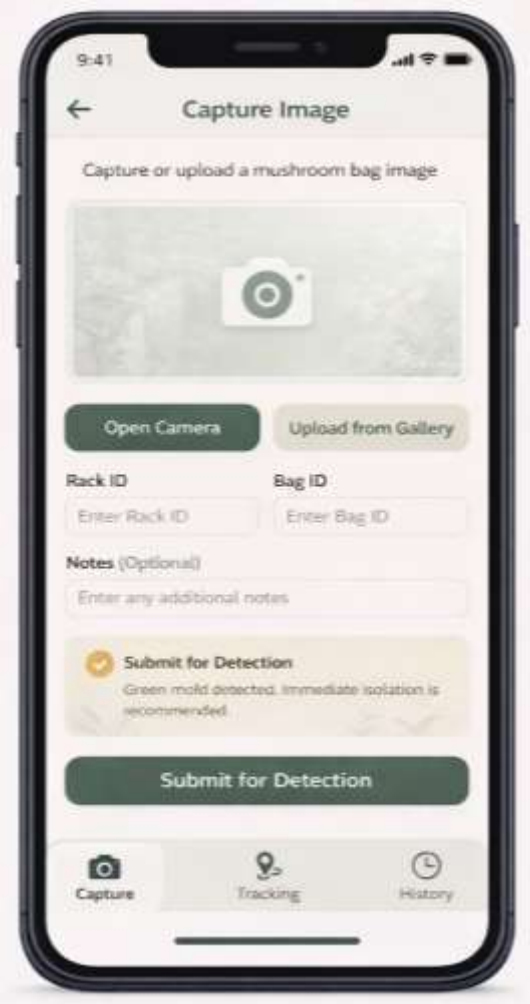


Figure 8 Disease Detection Main Page

- This page allows users to capture or upload an oyster mushroom bag image, enter Rack ID and Bag ID, and submit the image for analysis. This screen includes a camera preview box, image upload options, and real-time feedback messages.



Figure 9 Disease Detection History Page

- This page displays past detection records with filtering and search capabilities, allowing users to view results based on disease type or rack identification.



Figure 10 Disease Detection Tracking Page

- This page provides a visual representation of rack status using a color-coded grid, along with summary statistics for healthy and infected bags.

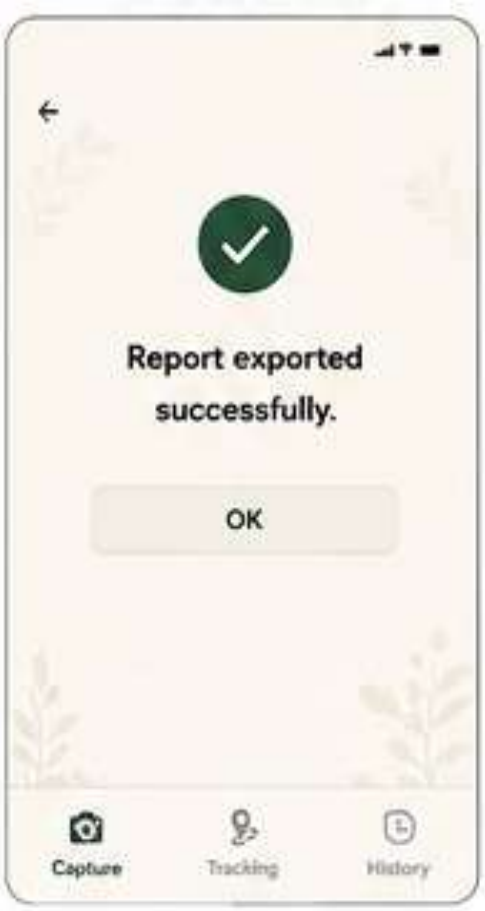


Figure 11 Report Export Successful Page

- This page provides confirmation when a report has been successfully generated and exported.

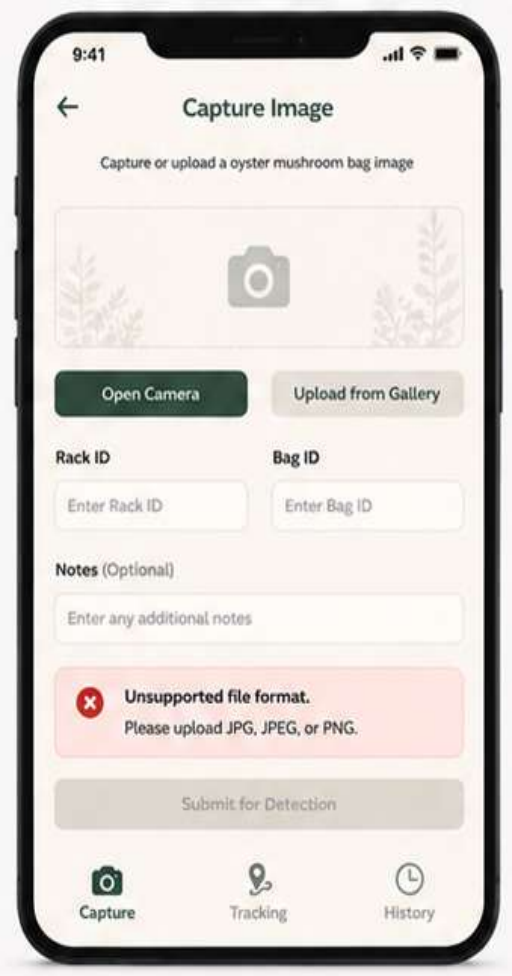


Figure 12 Unsupported File Format Error Page

- This page is displayed when an invalid image format is uploaded.

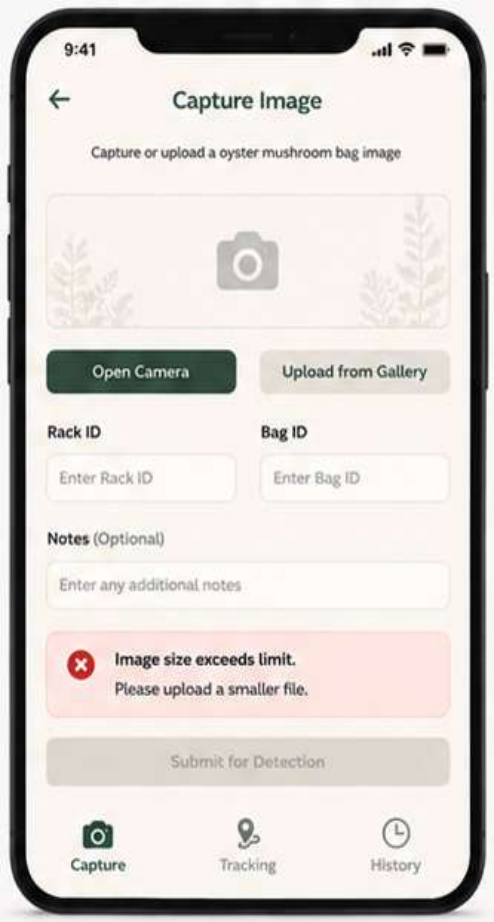


Figure 13 Image too Large Error Page

- This page appears when the uploaded file exceeds the size limit.

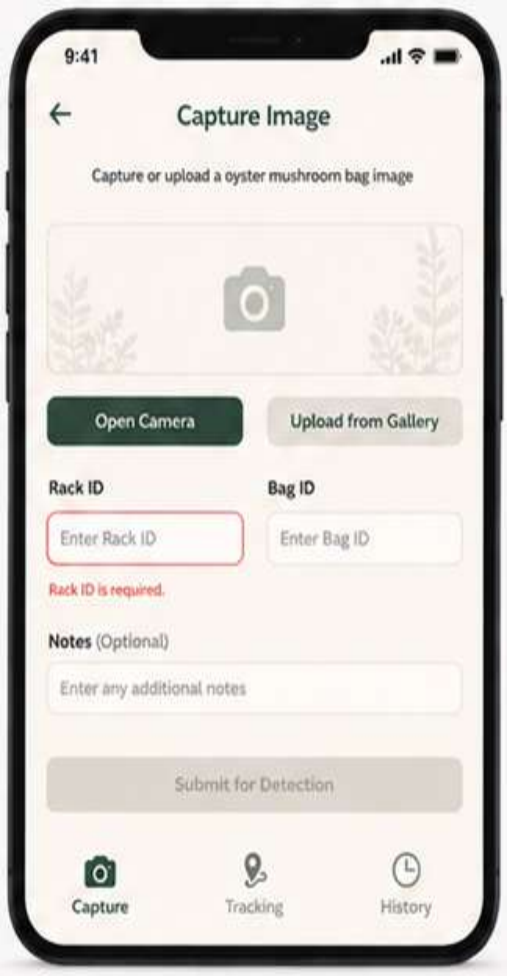


Figure 14 Rack ID Required Validation Page

- This page highlights missing required input fields.

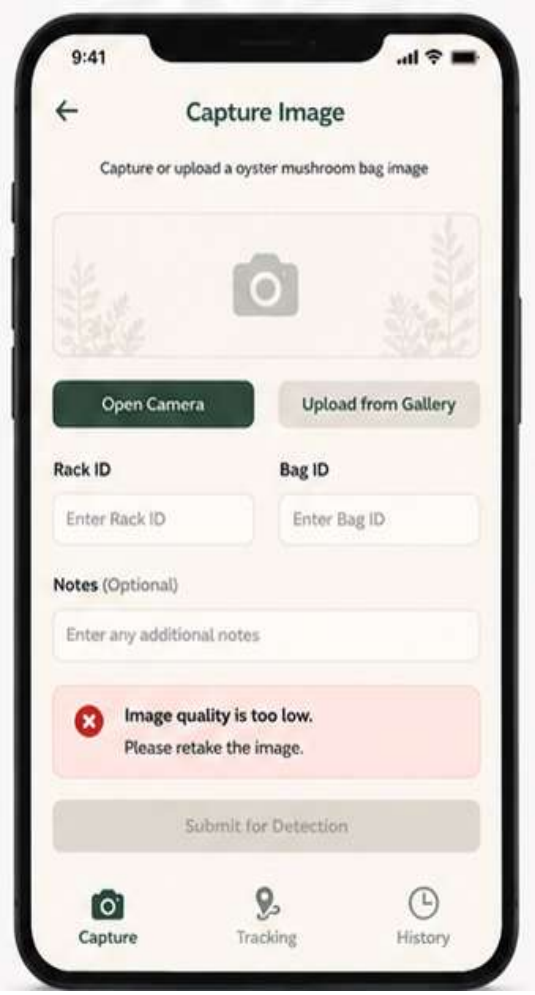


Figure 15 Low Image Quality Error Page

- This page prompts users to upload clearer images.

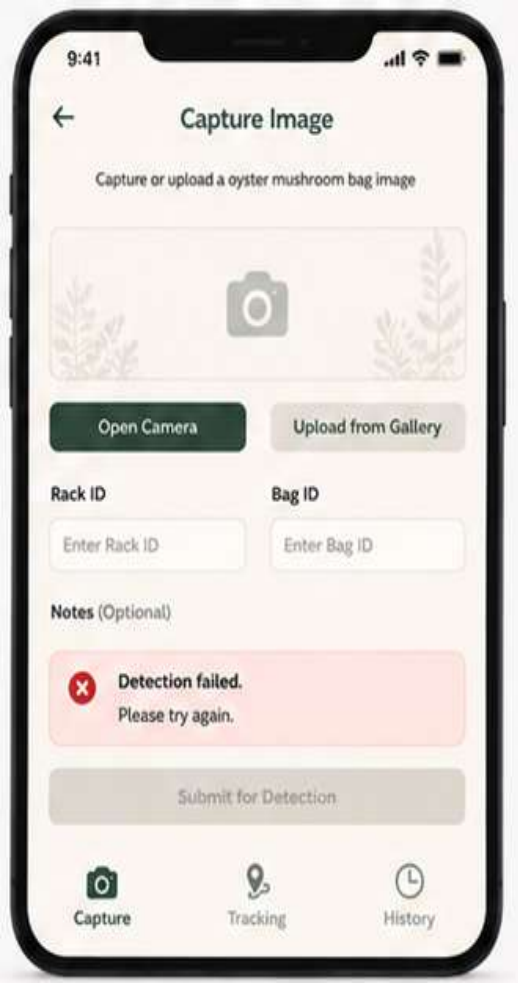


Figure 16 Detection Failed Error Page

- This page is displayed when the system is unable to process the image, providing users with the option to retry.

3.2 Hardware Interfaces

The hardware interfaces of the Rack-Level Micro-Climate & Growth Optimizer are designed to enable seamless communication between the software system and IoT-based sensing and actuation components used in oyster mushroom cultivation environments. The system is built around low-cost embedded hardware to ensure affordability, scalability, and real-time environmental control for farmers. The primary hardware controller used in the system is the ESP32, which acts as the central processing unit for all sensor data acquisition and device control operations. The ESP32 is connected to environmental sensing devices including the DHT11, which is used to measure ambient temperature and humidity levels inside each mushroom cultivation rack. These sensors continuously collect real-time environmental data, which is processed by the ESP32 and transmitted to the backend system using Wi-Fi communication.

The ESP32 communicates with the backend FastAPI server through RESTful HTTP requests using JSON-formatted data packets over standard TCP/IP protocols. The transmitted data includes humidity levels, temperature readings, and device status information, which are used by the system to monitor environmental conditions and make intelligent decisions regarding mushroom growth optimization. In addition to sensing, the hardware interface also supports actuation components such as mist sprayers, exhaust fans, and relay modules. The ESP32 receives control commands from the backend system and converts them into electrical signals to activate or deactivate these devices. This enables automatic regulation of humidity and ventilation based on real-time environmental conditions and mushroom growth stage predictions generated by the system.

The hardware system operates using Wi-Fi connectivity integrated within the ESP32 module, ensuring continuous communication with the backend server hosted on Ubuntu. In cases of temporary network interruption, the ESP32 can temporarily buffer sensor readings in its local memory and synchronize the data once the connection is restored, ensuring reliability and data consistency.

3.3 Software Interfaces

1. Supported Software Components

- Operating Systems - The mobile application is developed using React Native and operates on Android devices (version 10.0 and above), enabling farmers to remotely monitor environmental conditions, receive alerts, and control rack-level devices.
- Databases - The system uses MongoDB as the primary database to store user information, environmental sensor readings, humidity records, CO_2 concentration data, disease detection outputs, mushroom growth stage classifications, and device configurations.

2. Tools and Libraries

- React Native - React Native is used to develop the mobile application interface, providing real-time monitoring dashboards, notifications, and environmental control features.
- Fast API - Fast API is used as the backend framework to develop RESTful APIs, manage sensor communication, process real-time environmental data, and handle interactions between mobile applications, IoT devices, and machine learning services.
- Python, TensorFlow & OpenCV - Machine learning and computer vision functionalities are implemented using Python together with TensorFlow, OpenCV, and YOLOv8 for biological growth stage detection and fungal disease identification.
- Arduino IDE & ESP32 Libraries - The IoT sensing-and-actuation device is programmed using Arduino IDE and ESP32 communication libraries for interacting with humidity sensors, CO_2 sensors, mist sprayers, and exhaust fans.

3. APIs and Services

- REST API Services - REST APIs developed using Fast API to manage communication between the React Native mobile application, IoT devices, database, and machine learning services through JSON-based data exchange protocols.

- Machine Learning Services - AI services process mushroom images for growth stage classification and disease detection while analyzing CO_2 trends to predict physiological stress conditions.
- Cloud Database Services - Cloud database services manage environmental monitoring records, image analysis results, and real-time synchronization between the mobile application and backend server.

4. Data Flow and Communication

Incoming Data

- Environmental Sensor Inputs - The IoT device continuously collects humidity, temperature, and CO_2 concentration readings from rack-level sensors.
- Image Inputs - Images captured through the React Native mobile application are processed for mushroom growth stage detection and disease identification.
- User Inputs - Farmers can configure threshold values, monitor environmental conditions, and manage devices through the mobile application

Outgoing Data

- Environmental Control Commands - The system automatically activates mist sprayers and exhaust fans according to real-time environmental conditions.
- Alerts and Notifications - Real-time notifications are sent when abnormal humidity, high CO_2 levels, stress conditions, or fungal diseases are detected.
- Monitoring Dashboards - The React Native application displays environmental monitoring dashboards, growth stage updates, and disease analysis reports for efficient farm management.

5. Services and Communications

- Authentication Services - Secure authentication mechanisms ensure that only authorized users can access farm monitoring data and device controls.
- Artificial Intelligence Services - AI services perform mushroom growth stage classification, CO_2 stress prediction, and fungal disease detection using computer vision and deep learning techniques.
- IoT Communication Services - The ESP32-based sensing-and-actuation device communicates with the backend server through Wi-Fi and REST-based communication protocols to support real-time monitoring and automated environmental control.

3.4 Communications Interfaces

The following table defines the communication standards, data formatting, and synchronization mechanisms used between the hardware, backend, and mobile application components of the Rack-Level Micro-Climate & Growth Optimizer system.

Communication Method	Communication Standards	Security or Encryption	Data Transfer Rates	Synchronization Mechanisms
ESP32 to Fast API Backend Server	HTTP / HTTPS (REST API over Wi-Fi IEEE 802.11)	TLS/SSL (if HTTPS enabled), token-based request validation	Moderate (depends on Wi-Fi signal strength)	Periodic sensors push intervals (time-based polling and event-triggered updates)
Fast API Backend to Database (MongoDB)	Internal database driver communication (Mongo)	Authentication-based access control, role-based database permissions	High (low-latency internal server communication)	Immediate write/read operations with real-time data persistence
React Native Mobile App to Fast API Backend	HTTPS (REST API)	JWT authentication, TLS/SSL encryption	High (real-time dashboard updates)	Request/response model with periodic refresh and push-based notifications
ESP32 to Actuation Devices (Relay Modules, Fans, Mist Sprayer)	GPIO digital signals (HIGH/LOW voltage control)	Hardware-level isolation (relay protection circuits)	Instant (real-time switching)	Event-driven control triggered by backend commands or threshold conditions
Mobile App to Camera Module (Smartphone Hardware)	Native Android Camera API via React Native	Device-level permissions and secure sandboxing	High (depends on image size)	On-demand capture and upload to backend AI service
Fast API Backend to AI/ML Model Services	HTTP (REST API)	Internal API authentication, secured endpoints	Moderate (depends on model processing time)	Asynchronous processing with request/response callbacks

Table 1 Communication Interfaces

4. System Features

4.1. Moisture Stress Regulation Using Temperature-Humidity

4.1.1 Description and Priority

This system records measures of temperature and humidity obtained from the portable sensing device (which may be installed or used at rack level). These readings are used to determine a temperature-humidity based Transpiration Stress Index. The system is able to give corrective suggestions according to the calculated stress level and can turn on the mist sprayer to improve humidity, or the exhaust fan to decrease the humidity. This feature goes beyond static humidity thresholds and provides more meaningful rack-level micro-climate control.

4.1.2 Functional Requirements

- FR1 - Rack-level temperature shall be read from the portable sensor.
- FR2 - The system should gather the relative humidity level measurements from the portable sensor.
- FR3 - System shall save temperature and humidity data, along with the corresponding rack ID and timestamp.
- FR4 - The system shall measure temperature and humidity and use the combination to detect moisture stress on the racks.
- FR5 - Compute a Transpiration Stress Index based on temperature and humidity using sensor readings.
- FR6 - The system shall be able to determine the moisture condition by analyzing the stress value calculated.
- FR7 - The system should show the temperature, humidity and moisture stress status on the mobile application in a rack-wise manner.
- FR8 - If moisture is stressed above the set level the system shall issue an alert.
- FR9 - The system shall provide recommendations for corrective actions like turning on the mist sprayer or the exhaust fan.

4.1.3 Use Case Scenarios

Field	Description
Use case name	View Rack-Level Moisture Status
Primary Actors	Farmer/User
Pre-conditions	The farmer must be logged into the mobile application. The portable device and sensors must be active and connected.

Post-conditions	The farmer can view the latest moisture status of the selected rack. The sensor reading and moisture analysis result are saved in the system
Main Flows	**Action**
1	The farmer opens the mobile application.
2	The farmer selects the required farm, grow room, and rack.
3	The system requests the latest temperature and humidity readings from the portable device.
4	The portable device collects temperature and humidity values from the sensors.
5	The system stores sensor readings with the rack ID and timestamp.
6	The system analyzes the temperature-humidity interaction.
7	The system calculates the Transpiration Stress Index.
8	The system classifies the moisture condition as normal, low moisture stress, or high moisture stress.
9	The mobile application displays the rack-wise temperature, humidity, and moisture stress status to the farmer.

Table 2 Use Case of Moisture Stress Regulation Component

4.2 Disease Detection & Rack-Level Tracking

4.2.1 Description and Priority

The Disease Detection and Rack-Level Tracking feature enables early identification of fungal infections in oyster mushrooms using an AI-based image detection model (YOLOv8). The system allows users to capture or upload images of mushroom racks, analyze them for disease presence,

and track contamination at a rack and bag level. The system detects common diseases such as green mold (Trichoderma) and black mold (Aspergillus Niger). Once detected, the system stores records, and enables farmers to take corrective actions to prevent spreading.

This feature is High Priority as it directly impacts crop health, yield quality, and loss prevention. Failure of this module may result in delayed detection, leading to large-scale contamination and significant financial loss. The priority is further justified based on component ratings on a scale of 1 to 9, where the benefit is rated 9 due to its critical role in early disease detection and improving productivity, and the penalty is rated 9 as the absence of this feature can result in rapid disease spread and severe crop damage. The cost is rated 7, considering the need for AI model training, image processing, and computational resources, while the risk is rated 6, reflecting a moderate dependency on model accuracy and image quality, which can be managed through proper validation and dataset improvements.

4.2.2 Stimulus/Response Sequences

Stimulus	System Response
User captures or uploads an image of mushroom rack	System validates image format and uploads to backend
Image is received by system	System preprocesses image (resize, normalization)
Valid image is processed	YOLOv8 model detects objects/diseases
Disease detected in image	System identifies disease type and confidence score
No disease detected	System returns “Healthy” status
User enters rack ID / bag ID	System associates detection with specific rack
Detection result confirmed	System stores data in database
User requests history	System retrieves previous contamination records
Invalid image (blurred / unsupported format)	System shows error message and prompts re-upload
Model fails to process image	System logs error and requests retry

Table 3 Stimulus/Response Sequences of Disease Detection Component

4.2.3 Functional Requirements

- FR1 - The system shall allow users to capture images using a mobile device camera or upload images from local storage.
- FR2 - The system shall accept image formats including JPG, JPEG, and PNG.
- FR3 - The system shall validate input images for quality, format, and size before processing.
- FR4 - The system shall preprocess images (resizing, normalization) before feeding into the AI model.
- FR5 - The system shall detect diseases using a trained YOLOv8 object detection model.
- FR6 - The system shall identify at minimum, green mold (Trichoderma) and black mold (Aspergillus Niger)
- FR7 - The system shall display a “Healthy” status when no infection is detected.

- FR8 - The system shall allow users to input rack ID and bag ID.
- FR9 - The system shall associate detection results with the corresponding rack and bag.
- FR10 - The system shall store detection results in a database with timestamp.
- FR11 - The system shall allow users to view historical disease records per rack.
- FR12 - The system shall highlight infected racks visually in the dashboard.
- FR13 - The system shall handle invalid inputs by displaying appropriate error messages.
- FR14 - The system shall allow users to retry detection in case of processing failure.

4.2.4 Use Case Scenarios

Field	Description
Use Case Name	Capture and Detect Mushroom Disease
Primary Actor	Farmer/User
Pre-conditions	User is logged into the system; Camera/upload access is available
Post-conditions	Detection result is displayed and stored
Main Scenario Steps	**Action**
1	User opens disease detection feature
2	User captures or uploads an image
3	System validates image format and quality
4	System preprocesses image
5	System runs YOLOv8 model
6	System detects disease (if present)
7	System displays result
8	System stores detection result
Extension Steps	**Branching Action**
3a	If image format is invalid, system shows error
5a	If model fails, system requests retry
6a	If no disease detected, system displays “Healthy”

Table 4 Use Case 1 of Disease Detection Component

Field	Description
Use Case Name	Assign Detection to Rack and Bag
Primary Actor	Farmer/User
Pre-conditions	Detection process completed
Post-conditions	Detection linked to rack and bag
Main Scenario Steps	**Action**
1	System prompts user to enter rack ID and bag ID
2	User enters rack ID
3	User enters bag ID
4	System validates inputs

5	System associates detection with rack and bag
6	System stores updated record
Extension Steps	**Branching Action**
2a	If rack ID is missing, system prompts user
3a	If bag ID is invalid, system shows error

Table 5 Use Case 2 of Disease Detection Component

Field	**Description**
Use Case Name	View Disease Detection History
Primary Actor	Farmer/User
Pre-conditions	Detection records exist
Post-conditions	User views historical data
Main Scenario Steps	**Action**
1	User navigates to history
2	User selects rack filter
3	System retrieves records
4	System displays results
Extension Steps	**Branching Action**
2a	If no filter selected, system shows all records
3a	If no records found, system displays “No Data Available”

Table 6 Use Case 3 of Disease Detection Component

4.3 Biological Stage Detection & Harvest Time Calculation

4.3.1 Stimulus/Response Sequences

Stimulus	**System Response**
Portable device camera captures a scheduled or manual image of the mushroom rack.	Transmits the image to the backend system and triggers the Computer Vision (CV) processing pipeline.
System receives raw image data of fruiting bodies.	Performs image preprocessing (resizing/normalization) and runs a classification model to identify the current growth stage (e.g., Pinning).
Growth stage is successfully identified.	Updates the mobile application dashboard with the stage label and visualizes the detected features.
User requests an update on when to harvest.	Analyzes the size and morphology of the detected stage to compute an estimated "Days to Harvest" and displays it on the UI.

Detected growth stage changes (e.g., from Pinning to Fruiting).	Passes the new "Biological Context" to the Moisture Management Module to update the transpiration stress thresholds.
Image quality is too low (e.g., blurry or low light) for accurate classification.	Triggers a notification to the farmer via the mobile app requesting manual check or camera lens cleaning.

Table 7 Stimulus/Response Sequences of Biological Stage Detection Component

5. Other Non-functional Requirements

5.1 Performance Requirements

- Real-Time Environmental Monitoring: The system shall collect and transmit sensor data (humidity, temperature, and CO_2 levels) from the ESP32 and DHT11 to the backend system with a maximum delay of 2 seconds under stable Wi-Fi conditions.
- AI-Based Detection: The computer vision model shall identify mushroom growth stages and disease conditions (e.g., green mold, black mold) with an accuracy of at least 90%, and inference results shall be returned within 5 seconds per image under normal processing load.
- Environmental Control Response Time: The system shall activate or deactivate actuators (mist sprayers and exhaust fans) within 2 seconds after receiving threshold-based control commands from the backend system.
- Backend Processing (Fast API): The Fast API backend shall process incoming sensor data, manage API requests, and deliver responses within 3 seconds under standard operating conditions.
- Mobile Application Performance (React Native): The React Native application shall update dashboards, alerts, and real-time environmental data within 2–3 seconds of backend updates.
- Database Response Time: The MongoDB shall perform read/write operations for sensor data and monitoring records within 2 seconds, ensuring real-time synchronization across the system.

5.2 Safety Requirements

- Environmental Safety (Crop Protection): The system shall continuously monitor humidity, temperature, and CO_2 levels using the ESP32 and DHT11, and automatically trigger alerts when conditions exceed safe thresholds to prevent mushroom crop damage, contamination, or yield loss.
- Actuator Safety Control: The system shall ensure safe operation of hardware devices such as mist sprayers and exhaust fans by enforcing controlled activation limits through the backend system, preventing over-humidification, overheating, or continuous hardware operation that may damage equipment.
- Data Integrity & Access Control: The system shall enforce secure authentication and role-based access control within the FastAPI backend to ensure that only authorized users can access sensor data, modify thresholds, or control farm devices, preventing unauthorized or accidental system interference.

- AI Decision Safety: The computer vision and machine learning modules shall provide growth stage and disease detection results as advisory outputs only, ensuring that farmers are informed that final decisions regarding crop treatment and harvesting remain under human supervision.
- System Failure Safety: In case of network failure or backend unavailability, the ESP32-based hardware system shall continue local monitoring and maintain last known safe operating conditions until communication is restored, preventing sudden uncontrolled environmental changes.

5.3 Security Requirements

- Input Validation & Sensor Data Protection: The system must validate all incoming data from the ESP32 and DHT11 to ensure that only correctly formatted environmental readings (humidity, temperature, CO_2 values) are accepted, preventing corrupted, spoofed, or malicious data injection into the system.
- Authentication & Access Control: Secure user authentication shall be implemented in the Fast API backend to ensure that only authorized users can access the system. Role-based permissions will restrict control functions such as threshold adjustment, device activation, and data viewing to authorized users only.
- Data Encryption: All communication between the React Native mobile application, backend server, IoT devices, and database must be secured using HTTPS/TLS encryption to protect environmental data, user credentials, and control commands during transmission.
- Database Security & Isolation: The MongoDB database must enforce strict access control policies to ensure that sensor data, user records, and device configurations are only accessible to authenticated users and are logically separated to prevent unauthorized cross-access between farm systems.
- Device Communication Security: All ESP32-to-server communication must use secure token-based authentication or API key validation to prevent unauthorized devices from sending fake environmental readings or receiving control commands.

5.4 Software Quality Attributes

- Usability - The system must provide a simple and user-friendly mobile interface developed using React Native, designed for farmers with varying levels of technical knowledge. It should use clear dashboards, visual indicators for humidity and CO_2 levels, and easy-to-understand alerts for disease detection and growth stage changes, ensuring smooth interaction with the system.
- Reliability - The system must ensure continuous monitoring of environmental conditions using ESP32 and DHT11, even during temporary network failures. If the backend system experiences downtime, local sensor data collection and device control should continue, with automatic synchronization once the connection is restored.
- Maintainability - The backend developed using FastAPI must follow a modular and well-structured architecture, separating IoT communication, AI processing, and database operations. This ensures easy debugging, updates, and future enhancement of features such as disease detection and environmental optimization.

- Scalability - The system must support multiple cultivation racks and farms by scaling backend services and database operations efficiently. The architecture should handle increasing numbers of sensor nodes, image uploads, and real-time monitoring requests without performance degradation.
- Interoperability - The system must seamlessly integrate hardware and software components, including ESP32 IoT devices, mobile applications, and cloud databases. It should support standard communication protocols such as HTTP/HTTPS and JSON-based data exchange to ensure smooth interaction between all components.
- Availability (Target: 99.9%) - The system should maintain high availability using cloud-based backend services and continuous monitoring of IoT devices. Critical services such as data logging, alert generation, and environmental control must remain accessible with minimal downtime.
- Robustness - The system must be able to handle unstable network conditions commonly found in rural farming environments. The ESP32 should temporarily store sensor readings during disconnection and automatically resend data when connectivity is restored.

5.5 Business Rules

- **BR1 -** The system must ensure that only authorized users can access and control environmental monitoring devices connected through the ESP32, preventing unauthorized modification of humidity or ventilation settings in the mushroom farm.
- **BR2 -** All environmental readings collected from the DHT11 must be validated before being stored in the system. Any abnormal, missing, or corrupted sensor values must be rejected or flagged for review.
- **BR3 -** The system shall automatically trigger mist sprayers and exhaust fans based on predefined humidity and temperature thresholds, ensuring optimal conditions for oyster mushroom growth without requiring manual intervention.
- **BR4 -** The growth stage detection and disease identification outputs generated by machine learning models shall be advisory only. Final decisions regarding crop treatment and harvesting must be made by the farmer, not the system.
- **BR5 -** All sensor data, image analysis requests, and control commands must be processed through the FastAPI backend, ensuring centralized control, validation, and secure communication between system components.
- **BR6 -** The system must continuously monitor environmental conditions in real time, and any critical changes in humidity, temperature, or CO_2 levels must trigger immediate alerts to the mobile application.
- **BR7 -** All environmental data, disease detection results, and growth stage records must be stored securely in a centralized database with strict access control to prevent unauthorized access or modification.
- **BR8 -** In the event of network failure, the ESP32-based hardware system must continue local monitoring and retain the last known safe operational state until connectivity is restored.
- **BR9 -** If environmental parameters exceed safe cultivation limits, the system must immediately notify the user through the mobile application and automatically initiate corrective actions where applicable.

- **BR10 -** The system must always prioritize crop safety and environmental stability over automation efficiency, ensuring that no control action negatively impacts mushroom growth conditions.

6. Other Requirements

Req ID	*Other Requirement*
OR-01	The mobile application shall support a simple user-friendly interface with minimal technical complexity.
OR-02	The system shall support secure user authentication and role-based access for farmers and admins.
OR-03	The system shall protect farm-level information and avoid unnecessary disclosure of identifiable farm details.
OR-04	The system shall store image data, sensor readings, prediction results, alerts, and action logs in a structured database.
OR-05	The system shall support rack-level historical tracking for environmental readings, stress events, growth stages, and disease conditions.
OR-06	The system shall provide alerts for high moisture stress, and abnormal CO2 stress.
OR-07	The system shall provide cloud backup or scheduled database backup for important records.
OR-08	The system shall operate with commercially available sensors such as temperature, humidity, and CO2 sensors.
OR-09	The system shall support portable device integration with camera, sensors, mist sprayer, and exhaust fan control.

Table 8 Other Requirements

Appendix A: Glossary

Term	*Definition*
Oyster Mushroom	An edible mushroom species cultivated commercially in Sri Lanka and other countries.
Rack-Level Monitoring	Monitoring environmental and image data separately for each cultivation rack.
Micro-Climate	Localized environmental conditions such as temperature, humidity, and CO2 around a specific rack or shelf.
Biological Growth Stage	The development stage of mushrooms, such as early growth, pinning, fruiting, or maturity.
TSI	Transpiration Stress Index: a temperature-humidity based index used to evaluate effective moisture stress.
CO2 Stress Index	A normalized index calculated from CO2 readings to identify physiological stress conditions.

Computer Vision	A field of artificial intelligence used to analyze and classify image data.
Green Mold	A common fungal contamination issue that can affect mushroom cultivation.
Black Mold	A fungal contamination condition that can reduce mushroom quality and yield.
Portable Device	A device containing sensors, camera, and optional actuation control for rack-level monitoring.
Actuator	A hardware component such as a mist sprayer or exhaust fan that performs a physical action.
Image-Based Validation	Using captured images to confirm a condition detected from sensor trends.

Table 9 Glossary

Appendix B: Analysis Models

State –Transition Diagram

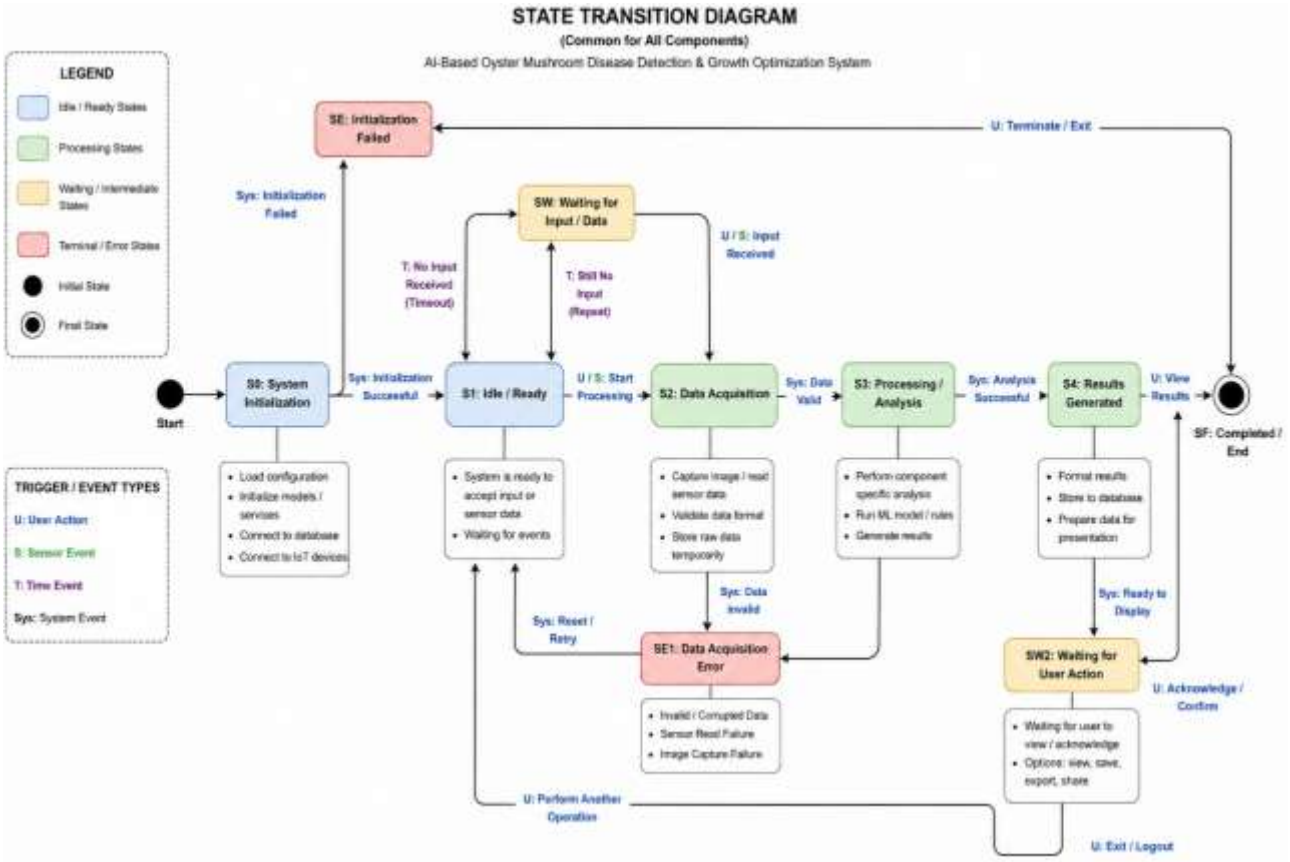


Figure 17 State Transition Diagram of the System

General Notes:

1. This state transition diagram represents the common workflow applicable to all four components (Stage detection, Micro-climate monitoring, CO2 stress prediction, and Disease detection).
2. Each component performs its specific processing during state S3: Processing/Analysis.
3. States starting with 'S' are standard states, 'SW' are waiting states, 'SE' are error states, and 'SF' is the final state.
4. The system returns to Idle (S1) after each operation to support continuous farm monitoring.

Data Flow Diagram

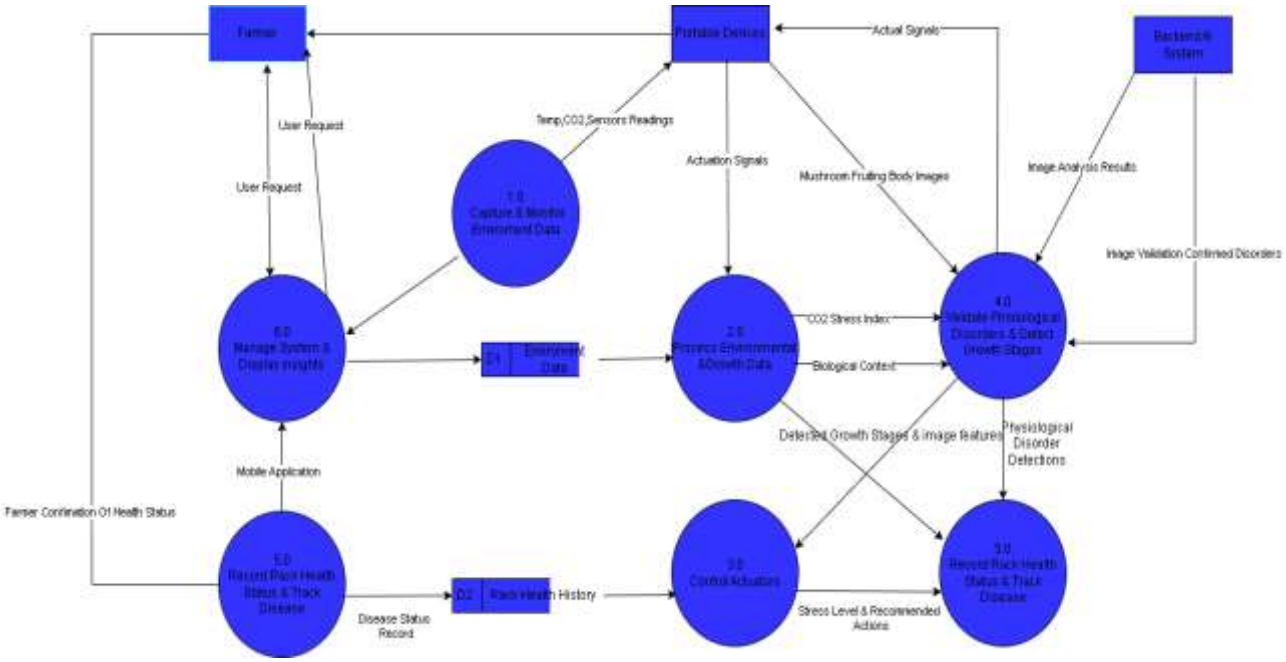


Figure 18 Data Flow Diagram of the System

Appendix C: To Be Determined List

Item	Description
TBD-01	Final TSI formula and threshold values after domain validation.
TBD-02	Final CO2 stress threshold values and trend duration rules.
TBD-03	Final portable device communication method between hardware and backend.